Отчет по заданию

Введение

В задании необходимо проанализировать работу сети, проверить корректность вычисления градиентов и исследовать её уязвимость к adversarial examples.

Содержание отчета:

- Описание реализованных функций
- Результаты численной проверки полученных градиентов
- Результаты атаки
- Выводы об уязвимости нейросети

Реализованные функции

1. Forward pass

Функция реализует прямой проход двухслойной нейронной сети нейронной сети, формируя её предсказание на основе весов и входных данных.

Формулы, которые реализует функция:

Скрытый слой:

$$Z_1 = XW_1^T + b_1$$

Применение функции активации ReLU:

$$A_1 = \mathrm{ReLU}(Z_1) = \max(0, Z_1)$$

Выходной слой:

$$Z_2 = A_1 W_2^T + b_2$$

Применение сигмоиды:

$$\hat{y} = \sigma(Z_2) = \frac{1}{1 + e^{-Z_2}}$$

2. Функции активации

В данной нейронной сети применяются две функции активации:

ReLU

Данная функция активации действует в скрытом слое и применяется внутри функции forward. Функция может работать как с одним примеров, так и с матрицей. Соответственно, вход в ReLU может быть вектором $(1, 8)$, и может быть матрицей $(N, 8)$.

Реализует формулу:

$$\mathrm{ReLU}(x) = \max(0, x)$$

Функция производной ReLU

Реализует формулу:

$$\mathrm{dReLU}(x) = \begin{cases} 0 & \text{если } x \leq 0 \\ 1 & \text{если } x > 0 \end{cases}$$

Сигмоида

Функция действует на выходном слое и также применяется внутри forward. Реализует формулу:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

3. Функция потерь

В задаче используется бинарная кросс-энтропия (BCE) в качестве функции потерь.

Функция в программе принимает на вход истинное решение y и предсказание модели $\hat{y}$ и реализует формулу:

$$L = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y}))$$

Функция возвращает вектор потерь для каждого конкретного примера.

4. Compute_gradients

Функция вычисляет аналитические градиенты бинарной кросс-энтропии по всем параметрам сети: $\frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial b_1}, \frac{\partial L}{\partial W_2}, \frac{\partial L}{\partial b_2}$.

Формулы, которые реализует функция:

Сначала с помощью функции forward вычисляются предсказания нейросети. Дальше считаем градиенты используя полученные данные:

$$d_2 = \hat{y} - y \in \mathbb{R}^{N \times 1}$$

$$\text{grad_W2}[i] = d_2[i] \cdot a_1[i] \in \mathbb{R}^{1 \times 8} \quad \text{для каждого объекта } i$$

$$\text{grad_b2}[i] = d_2[i] \in \mathbb{R}^1 \quad \text{в коде } d_2.reshape(N, 1, 1)$$

$$d_1 = (d_2 W_2) \odot \mathrm{ReLU}'(Z_1) \in \mathbb{R}^{N \times 8}$$

$$\text{grad_W1}[i] = d_1[i]^T \cdot X[i] \in \mathbb{R}^{8 \times 2} \quad \text{в коде } d_1.reshape(N, 8, 1) * X.reshape(N, 1, 2)$$

$$\text{grad_b1}[i] = d_1[i] \in \mathbb{R}^{8 \times 1} \quad \text{в коде } d_1.reshape(N, 8, 1)$$

В итоге функция возвращает матрицы градиентов размерностей:

- grad_W1 - (N, 8, 2)
- grad_b1 - (N, 8, 1)
- grad_W2 - (N, 1, 8)
- grad_b2 - (N, 1, 1)

5. Input_gradients

Функция input_gradient вычисляет градиент функции потерь по входным признакам. То есть она позволяет найти направление изменения входного вектора, которое увеличивает значение функции потерь, что необходимо для функции PGD-атаки.

Функция реализует формулу:

$$\text{inp_grad} = \frac{\partial L}{\partial x} = (d_2 \cdot W_2) \odot \text{ReLU}'(Z_1) \cdot W_1$$

Функция корректно работает как для одного примера, так и для батча данных.

6. Gradient_check

Функция gradient_check проверяет аналитические градиенты, посчитанные в compute_gradients, с помощью метода конечных разностей.

Алгоритм работы:

1. Для каждого элемента веса или смещения создаются два слегка изменённых варианта параметра: $w_i + \varepsilon$ и $w_i - \varepsilon$, где $\varepsilon = 10^{-5}$.

2. Вычисляется функция потерь для этих параметров L_+ и L_-.

3. Численный градиент:
$$\frac{\partial L}{\partial w_i} \approx \frac{L_+ - L_-}{2\varepsilon}$$

4. Сравнивается численный градиент с аналитическим, вычисляются максимальная абсолютная и относительная ошибка:
$$\text{max_abs_diff} = \max |\text{grad_numeric} - \text{grad_analytical}|,$$
$$\text{max_rel_diff} = \max \frac{|\text{grad_numeric} - \text{grad_analytical}|}{|\text{grad_analytical}| + 10^{-9}}$$

5. Проверка успешна, если $\text{max_abs_diff} < 10^{-4}$ или $\text{max_rel_diff} < 10^{-5}$.

Функция поддерживает проверку как одного примера, так и батча, возвращая словарь с результатами проверки для каждого параметра сети.

Внутри описана функция check_parameters, которая для каждого конкретного параметра вычисляет max_abs_diff, max_rel_diff. На основе этих величин функция формирует результат проверки и возвращает словарь, содержащий эти данные.

Сама функция gradient_check возвращает словарь, содержащий словари, которые вернула check_parameters для каждого конкретного параметра.

7. PGD_attack

Функция pgd_attack реализует итеративную генерацию adversarial examples для бинарной классификации с помощью метода Projected Gradient Descent (PGD). Функция ищет минимальное по норме ℓ_2 возмущение δ для каждого правильно классифицированного примера x, такое что сеть меняет своё предсказание:

$$\min_{\delta} \|\delta\|_2 \quad \text{при условии } \hat{y}(x + \delta) \neq y$$

Алгоритм работы функции:

1. Определяются правильно классифицированные примеры (`correct_mask`).
2. Для каждого правильного примера инициализируется возмущение
$$\delta = 0.$$
3. На каждом шаге (до `steps`) вычисляется градиент функции потерь по входу с помощью `input_gradient`.
4. Возмущение обновляется по правилу:
$$\delta \leftarrow \delta + \mathrm{lr} \cdot \frac{\partial L}{\partial x}.$$
5. Если предсказание сети меняется, атака считается успешной, и итерации прекращаются.

Функция возвращает:

- dlts — матрица найденных возмущений для каждого примера
- success — булев массив успешных атак
- correct_mask — булев массив правильно классифицированных примеров

Результаты численной проверки градиентов

Проверка осуществлялась в функции gradient_check методом конечных разностей на подвыборке из рандомно выбранных 50 элементов.

Параметр	max_abs_diff	max_rel_diff	Статус
W1	1.94×10^{-10}	2.45×10^{-8}	PASS
b1	4.34×10^{-11}	1.83×10^{-9}	PASS
W2	9.89×10^{-11}	8.97×10^{-8}	PASS
b2	1.45×10^{-11}	1.37×10^{-9}	PASS

Таким образом, аналитические градиенты совпадают с численными с достаточной точностью. Это подтверждает корректность реализации функции compute_gradients.

Результаты PGD-атаки

- Количество правильно классифицированных примеров: 474 из 500
- Количество успешных атак: 471
- Норма ℓ_2 возмущений δ по успешным атакам:
  - Минимальная: 0.2662
  - Медианная: 0.7906
  - Максимальная: 1.3301

Это показывает, что сеть оказалась уязвима к небольшим целенаправленным изменениям входа — почти все правильно классифицированные точки можно превратить в ошибки сети с помощью небольших возмущений. Это подтверждает, что нейронные сети чувствительны к adversarial examples даже при низкой размерности пространства признаков.

Ответы на теоретические вопросы к заданию

Визуализация атаки:

На scatter-plot показаны исходные точки, граница принятия решений сети и успешные adversarial examples. Почти все правильно классифицированные точки, за исключением совсем отдалённых от границы принятия решений, оказались подвержены атаке, однако величина минимального возмущения δ зависит от расстояния до границы решений:

- точки, лежащие ближе к границе, требуют меньших возмущений для изменения предсказания;
- точки, находящиеся дальше, всё равно поддаются атаке, но δ у них больше

Таким образом, нейронная сеть чувствительна к целенаправленным возмущениям практически для всех точек, но самые уязвимые точки расположены близко к границе принятия решений.

Уязвимость нейронных сетей к таким атакам:

Есть несколько причин, почему нейросети уязвимы к PGD-атакам. Одна из них, это локальная линейность нейросети. Это означает что в маленькой окрестности любого примера функция, которую реализует нейросеть может быть представлена в качестве линейной (например, функция ReLU, используемая в нашей нейросети в маленькой окрестности положительного значения x равна x). Это делает PGD-атаку эффективной, потому что градиент функции потерь по входным данным точно указывает направление, в котором можно изменить выход.

Еще одна причина связана с геометрией признаков, которая может сильно влиять на уязвимость нейросети. В задаче бинарной классификации классы могут располагаться близко друг к другу (как в нашем случае). Это означает что часто достаточно маленького возмущения, чтобы перейти границу, разделяющую два класса.

Таким образом, эти две проблемы приводят к тому, что у нас есть много уязвимых точек, лежащих близко к границе, и есть метод, позволяющий точно определить направление движения к этой границе. Это приводит к росту уязвимости нейросети.